

# PROBLEM 1 — AI/ML THEORY & AOROA PRIVATE LLM

---

(By Tojiboev Ikromjon )

## 1. Executive Summary

## 2. Q1 — Variational Inference, KL Divergence, ELBO, and the Diffusion Connection

- 2.1 Variational Inference
- 2.2 KL Divergence
- 2.3 Evidence Lower Bound (ELBO)
- 2.4 Reparameterization Trick
- 2.5 Connection to Diffusion Models
- 2.6 Conclusion
- References (Q1)

## 3. Q2 — Transformer Architecture, Attention, KV Cache, and GQA

- 3.1 Transformer Architecture
- 3.2 Self-Attention
- 3.3 Attention Equation
- 3.4 Why Scale by  $\sqrt{d_k}$
- 3.5 Multi-Head Attention
- 3.6 KV Cache
- 3.7 Grouped-Query Attention (GQA)
- 3.8 MHA vs MQA vs GQA
- 3.9 Engineering Interpretation for AOROA
- 3.10 Conclusion

## 4. Q3 — Diffusion Models vs GANs, Consistency Models, and Flow Matching

- 4.1 Generative Adversarial Networks (GAN)
- 4.2 Diffusion Models
- 4.3 Diffusion vs GAN Comparison
- 4.4 Why Reduce Diffusion Sampling Steps?
- 4.5 Consistency Models
- 4.6 Flow Matching

- 4.7 Consistency Models vs Flow Matching
- 4.8 Engineering Interpretation
- 4.9 Conclusion

## **5. Q4 — LoRA/QLoRA, RAG, Long Context, SFT, RLHF, and DPO**

- 5.1 Parameter-Efficient Fine-Tuning: LoRA
- 5.2 QLoRA
- 5.3 Retrieval-Augmented Generation (RAG)
- 5.4 RAG vs Fine-Tuning
- 5.5 Long Context
- 5.6 Hybrid Retrieval + Long Context
- 5.7 Supervised Fine-Tuning (SFT)
- 5.8 Reinforcement Learning from Human Feedback (RLHF)
- 5.9 Direct Preference Optimization (DPO)
- 5.10 SFT vs RLHF vs DPO
- 5.11 Engineering Trade-offs
- 5.12 Conclusion

## **6. Q5 — AOROA Private LLM Architecture and Evaluation**

- 6.1 Problem Context
- 6.2 Proposed Architecture
- 6.3 Why RAG?
- 6.4 Why LoRA / QLoRA?
- 6.5 RAG + LoRA/QLoRA
- 6.6 Evaluation Framework
- 6.7 Controlled Evaluation Dataset
- 6.8 A/B Evaluation
- 6.9 Security and Privacy
- 6.10 Recommended Development Strategy
- 6.11 Final Recommendation
- 6.12 Q5 Final Answer

## **7. Overall Technical Conclusion**

## **8. References**

## **Problem 1 — Executive Summary**

### **Executive Summary**

This report examines five core areas of modern AI/ML systems relevant to the design and deployment of private, domain-specific language-model solutions. It first reviews variational inference, KL divergence, the evidence lower bound (ELBO), variational autoencoders, and their probabilistic connection to diffusion models. It then analyzes the Transformer architecture, including self-attention, scaling, multi-head attention, KV caching, and grouped-query attention (GQA), with emphasis on inference efficiency.

The report next compares generative modeling approaches, focusing on the differences between GANs and diffusion models and discussing consistency models and flow matching as approaches for reducing generative sampling cost. It then examines practical methods for adapting and augmenting large language models, including LoRA/QLoRA, retrieval-augmented generation (RAG), long-context processing, supervised fine-tuning (SFT), RLHF, and DPO.

Finally, the report proposes a privacy-preserving architecture for AOROA Labs based on a private LLM combined with RAG and parameter-efficient adaptation. The proposed system separates knowledge adaptation from behavioral/task adaptation: RAG provides access to changing domain knowledge, while LoRA/QLoRA can adapt the model to domain-specific tasks and response behavior. The proposed architecture is evaluated using retrieval, answer quality, groundedness, citation correctness, hallucination, latency, resource usage, and task-completion metrics.

The overall recommendation is to develop the system incrementally, beginning with a private RAG baseline, evaluating its performance, and subsequently introducing parameter-efficient adaptation where the evaluation indicates a measurable benefit.

## 2. Q1 — Variational Inference, KL Divergence, ELBO, and the Diffusion Connection

### 2.1 Variational Inference

Variational inference (VI) converts an otherwise intractable posterior-inference problem into an optimization problem. Given observed data  $x$  and latent variables  $z$ , the target posterior is

$$p(z | x) = \frac{p(x, z)}{p(x)},$$

where the marginal likelihood

$$p(x) = \int p(x, z) dz$$

may be computationally intractable.

VI introduces a tractable variational distribution  $q_\phi(z | x)$  and seeks a distribution that approximates the true posterior:

$$q_\phi(z | x) \approx p(z | x).$$

The approximation is obtained by optimizing a divergence between the two distributions.

### 2.2 KL Divergence

The discrepancy between the variational distribution and the true posterior can be expressed using Kullback–Leibler (KL) divergence:

$$D_{KL} \left( q_\phi(z | x) \parallel p(z | x) \right) = \mathbb{E}_{q_\phi} \left[ \log \frac{q_\phi(z | x)}{p(z | x)} \right].$$

KL divergence measures how different the approximation is from the target distribution. Since the exact posterior is generally difficult to compute, directly minimizing this expression is also inconvenient. This motivates the ELBO formulation.

## 2.3 Evidence Lower Bound

Starting from the marginal likelihood,

$$\log p(x) = \log \int p(x, z) dz,$$

we introduce  $q_\phi(z | x)$ :

$$\log p(x) = \log \int q_\phi(z | x) \frac{p(x, z)}{q_\phi(z | x)} dz.$$

Applying Jensen's inequality gives the evidence lower bound:

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p(x, z) - \log q_\phi(z | x)].$$

Therefore,

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi(z|x)} [\log p(x, z) - \log q_\phi(z | x)]$$

is the **ELBO**.

It can also be decomposed as:

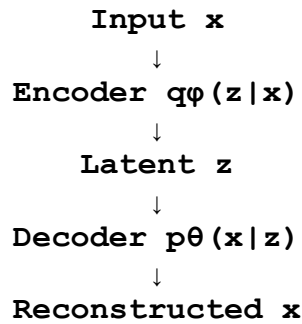
$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)} [\log p(x | z)] - D_{KL}(q_\phi(z | x) \parallel p(z)).$$

This gives the standard interpretation:

- **reconstruction term** — encourages the latent representation to explain the observed data;
- **KL regularization term** — encourages the learned latent distribution to remain close to the prior.

## 2.4 Variational Autoencoders

A Variational Autoencoder (VAE) applies this framework using an encoder and decoder.



The encoder approximates the posterior, while the decoder models the data-generation process.

A central implementation issue is that directly sampling from

$$\mathbf{z} \sim q_{\phi}(\mathbf{z} | \mathbf{x})$$

would prevent straightforward backpropagation through the stochastic sampling operation.

The **reparameterization trick** addresses this by expressing the latent variable as

$$\mathbf{z} = \mu_{\phi}(\mathbf{x}) + \sigma_{\phi}(\mathbf{x}) \odot \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

This separates the random component from the differentiable parameters, allowing gradient-based optimization.

## 2.5 Connection to Diffusion Models

The probabilistic structure extends naturally to diffusion models.

A diffusion model defines a sequence of latent noisy states:

$$\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{x}_T.$$

The forward process gradually adds noise, while the learned reverse process attempts to recover the data distribution.

Conceptually:

$$\begin{array}{c} \text{Forward process} \\ \mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \mathbf{x}_2 \rightarrow \dots \rightarrow \mathbf{x}_T \\ \text{adding noise} \\ \\ \text{Reverse process} \\ \mathbf{x}_T \rightarrow \mathbf{x}_{T-1} \rightarrow \dots \rightarrow \mathbf{x}_1 \rightarrow \mathbf{x}_0 \\ \text{learned denoising} \end{array}$$

The connection to variational inference arises because the forward noising process can be treated as a fixed probabilistic path, while the reverse process is parameterized as a learned generative model. The diffusion objective can be derived from a variational bound, connecting denoising training to probabilistic latent-variable modeling.

The commonly used noise-prediction objective is therefore not an unrelated heuristic: it is a practical parameterization of the underlying probabilistic training framework.

## 2.6 Q1 Conclusion

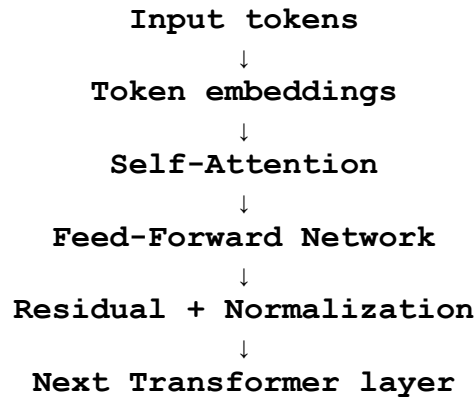
Variational inference converts otherwise intractable posterior inference into an optimization problem by fitting a tractable variational distribution to the true posterior. KL divergence provides the discrepancy measure, while the ELBO provides a tractable lower bound on the log marginal likelihood. VAEs use this framework with an amortized encoder and reparameterization trick. Diffusion models extend the same probabilistic principle to a sequence of latent noisy states: the forward noising process acts as a variational distribution, while the learned reverse process defines the generative model. The diffusion training objective can therefore be derived from a variational bound, and the commonly used noise-prediction objective is a simplified parameterization of this probabilistic training framework.

## 3. Q2 — Transformer Architecture, Attention, KV Cache, and GQA

### 3.1 Transformer Architecture

The Transformer is an attention-based neural architecture designed to process sequences without relying on recurrent computation. Its central mechanism is **self-attention**, which allows each token to incorporate information from other tokens in the sequence.

A simplified Transformer flow is:



For large language models, Transformer layers are repeatedly applied to produce contextual representations and, during autoregressive generation, predict subsequent tokens.

## 3.2 Self-Attention

For an input representation  $X$ , the Transformer generates three projections:

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned}$$

where:

- $Q$ = queries
- $K$ = keys
- $V$ = values
- $W_Q, W_K, W_V$ = learned projection matrices.

The query of one token is compared with the keys of other tokens to determine how strongly information from each position should contribute.

The resulting attention representation is:



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

### 3.3 Why Scale by $\sqrt{d_k}$ ?

The dot product  $QK^T$  tends to increase in magnitude as the key/query dimensionality  $d_k$  increases.

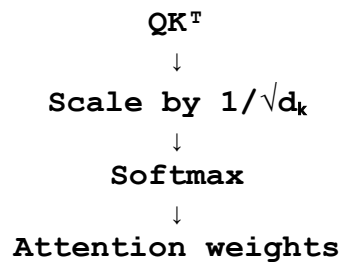
Without scaling, the resulting logits can become large, causing the softmax distribution to become excessively concentrated. This can produce very small gradients and make optimization less stable.

The scaling term:

$$\frac{1}{\sqrt{d_k}}$$

normalizes the magnitude of the dot products before the softmax operation.

Therefore:

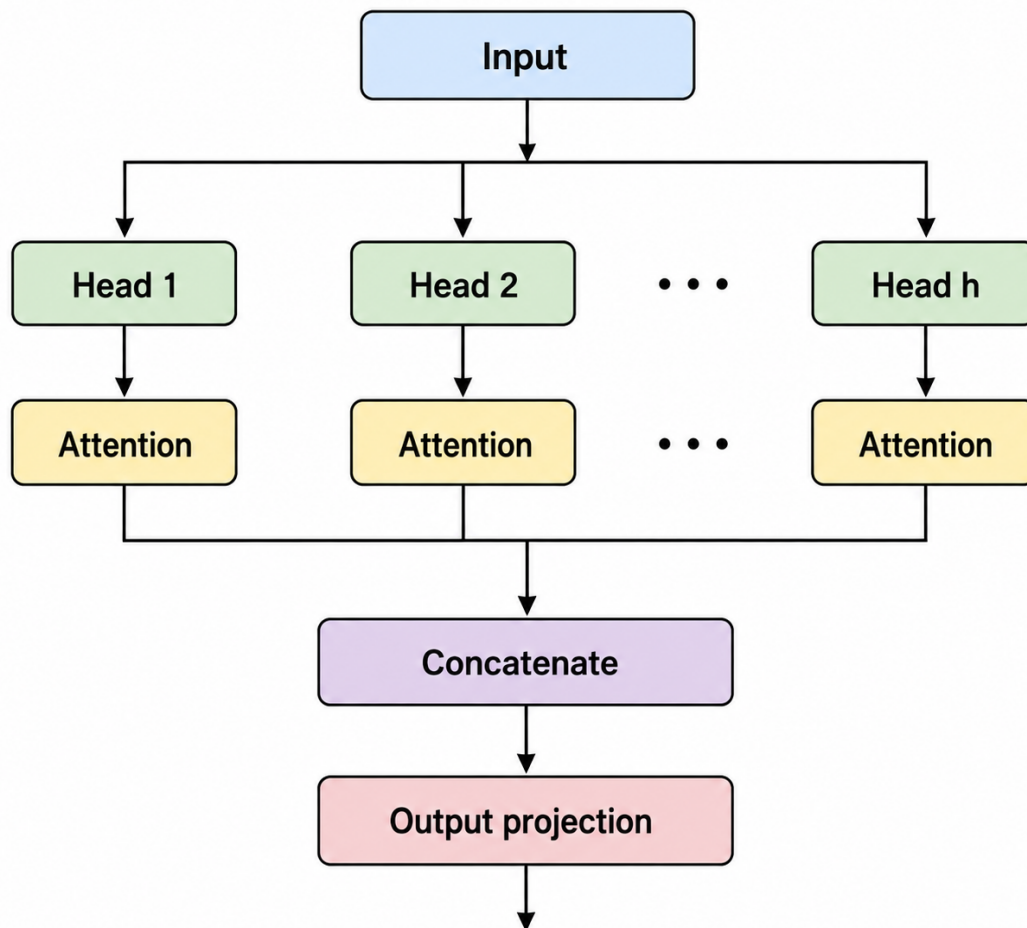


is an important part of scaled dot-product attention.

### 3.4 Multi-Head Attention

Instead of computing attention once, Multi-Head Attention divides the representation into multiple attention heads.

Conceptually:



Each head can learn different relationships between tokens. The outputs are concatenated and projected back into the model representation space.

This allows the model to capture multiple interaction patterns simultaneously.

## 3.5 KV Cache

During autoregressive generation, the model generates tokens sequentially:

Token 1  
↓  
Token 2  
↓  
Token 3  
↓  
Token 4  
↓  
...

At each decoding step, previously computed key and value representations can be reused.

The **KV cache** stores these previous *K* and *V* representations.

Without caching:

Every new token  
↓  
Recompute previous K/V  
↓  
More computation

With KV caching:

Previous tokens  
↓  
Stored K/V  
↓  
Reuse  
+  
New token K/V

This substantially reduces redundant computation during autoregressive decoding.

However, KV caching introduces a memory cost because the stored key/value tensors grow with:

- sequence length,
- number of layers,
- number of KV heads,
- head dimension,
- batch size.

Therefore, for large language model serving, KV-cache memory can become an important bottleneck.

### 3.6 Grouped-Query Attention (GQA)

Grouped-Query Attention addresses the KV-cache memory problem by allowing multiple query heads to share key/value heads.

A simplified comparison is:

Multi-Head Attention

$$\begin{aligned} Q_1 &\rightarrow K_1 V_1 \\ Q_2 &\rightarrow K_2 V_2 \\ Q_3 &\rightarrow K_3 V_3 \\ Q_4 &\rightarrow K_4 V_4 \end{aligned}$$

Grouped-Query Attention

$$\begin{array}{l} Q_1 \quad \neg \\ Q_2 \quad \neg \rightarrow K_1 V_1 \\ Q_3 \quad \neg \\ Q_4 \quad \neg \end{array}$$

Thus, GQA maintains multiple query heads while using fewer key/value heads.

This reduces the amount of  $K/V$  information that must be stored in the cache.

The design therefore provides an important engineering trade-off:

MHA → maximum KV capacity

while

GQA → fewer KV heads → lower KV-cache memory

with the goal of maintaining strong model quality.

### 3.7 MHA vs MQA vs GQ

| Architecture | Query heads | KV heads     | KV-cache memory |
|--------------|-------------|--------------|-----------------|
| <b>MHA</b>   | Many        | Many         | Highest         |
| <b>MQA</b>   | Many        | 1            | Lowest          |
| <b>GQA</b>   | Many        | Fewer groups | Intermediate    |

This creates a practical serving trade-off.

**MHA** provides independent key/value representations for every attention head but requires more KV-cache memory.

**MQA** aggressively reduces KV memory by sharing one key/value head across query heads.

**GQA** provides an intermediate design by grouping query heads and sharing key/value heads among each group.

For production LLM inference, GQA can therefore be attractive when memory efficiency and decoding throughput are important while retaining a richer attention structure than MQA.

## 3.8 Engineering Interpretation for AOROA

For a private LLM deployment, attention architecture is not only a theoretical choice. It affects:

- GPU memory requirements,
- KV-cache capacity,
- inference latency,
- concurrent users,
- context length,
- serving cost.

The relevant engineering objective is therefore not simply:

**“Use the largest possible model.”**

Instead:

**Select an architecture that provides an appropriate quality–memory–latency trade-off for the target workload.**

For a private logistics-oriented LLM, this becomes particularly relevant when the system needs to process long operational contexts or serve multiple users concurrently.

## 3.9 Q2 Conclusion

The Transformer uses scaled dot-product self-attention to dynamically model relationships between tokens. The  $1/\sqrt{d_k}$  scaling factor controls the magnitude of attention logits and supports stable softmax behavior. Multi-head attention allows the model to learn multiple interaction patterns, while KV caching avoids recomputing previous key and value representations during autoregressive decoding. However, KV-cache memory grows with sequence length and the number of KV heads. Grouped-Query Attention addresses this bottleneck by allowing multiple query heads to share fewer key/value heads, providing a practical trade-off between attention capacity and inference memory efficiency. For private LLM deployment, these architectural choices directly affect latency, GPU memory, context capacity, concurrency, and serving cost.

## 4. Q3 — Diffusion Models vs GANs, Consistency Models, and Flow Matching

### 4.1 Generative Adversarial Networks

Generative Adversarial Networks (GANs) use two neural networks trained in competition:

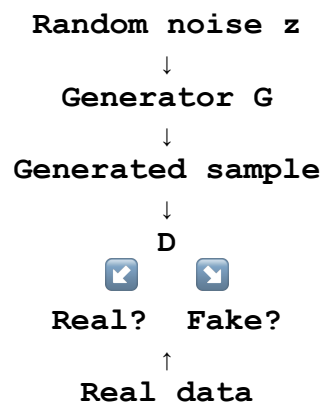
- a **generator**  $G$ , which produces synthetic samples;
- a **discriminator**  $D$ , which distinguishes generated samples from real data.

The basic adversarial objective is:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))].$$

The generator attempts to produce samples that appear realistic to the discriminator, while the discriminator learns to distinguish real and generated samples.

Conceptually:



GANs can provide very fast generation after training because generating a sample generally requires a single forward pass through the generator.

However, adversarial optimization can be difficult to stabilize. One important failure mode is **mode collapse**, where the generator produces insufficiently diverse samples.

## 4.2 Diffusion Models

Diffusion models use a fundamentally different generation mechanism.

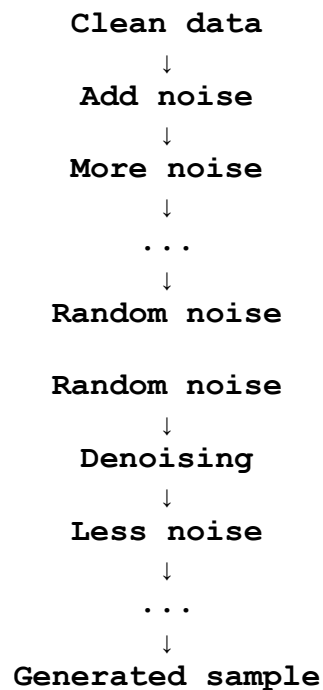
The forward process gradually adds noise to data:

$$x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T.$$

The model then learns the reverse denoising process:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.$$

Conceptually:



Left: 23 O<sub>2</sub> Right: 9 O<sub>2</sub>, Net diffusion Random motion carries molecules both ways, but more move from the more concentrated side to the less concentrated side. Initial molecules on left O<sub>2</sub> Initial molecules on left , Learn more a commonly used training formulation asks a neural network to predict the noise added to a sample. The model therefore learns how to reverse the corruption process.



Compared with GANs, diffusion models generally offer a more stable likelihood-based/probabilistic training framework, but traditional sampling requires multiple denoising steps.

### 4.3 Diffusion vs GAN

| Property           | GAN                           | Diffusion                       |
|--------------------|-------------------------------|---------------------------------|
| Training mechanism | Adversarial                   | Denoising                       |
| Main components    | Generator + discriminator     | Denoising model + noise process |
| Sampling           | Usually very fast             | Traditionally multi-step        |
| Training stability | Can be difficult              | Generally more stable           |
| Diversity          | Can suffer from mode collapse | Strong diversity                |
| Main challenge     | Adversarial optimization      | Sampling cost                   |

The central trade-off can therefore be summarized as:

GAN → fast generation

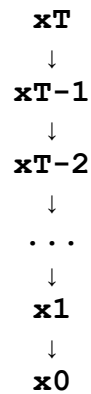
while

Diffusion → high-quality iterative generation.

This does not mean that one approach is universally superior. The appropriate choice depends on the required quality, diversity, latency, and computational budget.

## 4.4 Why Reduce Diffusion Sampling Steps?

A traditional diffusion model may require many denoising iterations:



Each step requires model inference.

Consequently:

more sampling steps  $\Rightarrow$  higher inference cost and latency.

This motivates research into methods that can generate high-quality samples with substantially fewer steps.

Two relevant approaches are **consistency models** and **flow matching**.

## 4.5 Consistency Models

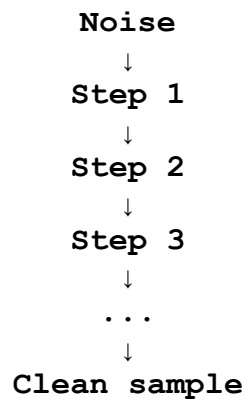
Consistency models are designed to enable efficient generation using significantly fewer sampling steps.

The basic intuition is that points along an appropriate diffusion trajectory should map consistently toward the same underlying clean sample.

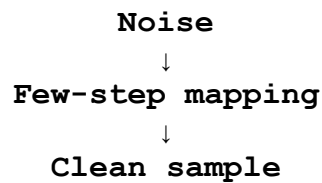
Instead of requiring the model to perform a long sequence of small denoising transitions, a consistency model learns a mapping that can move from a noisy state toward the corresponding clean output using a small number of evaluations.

Conceptually:

**Traditional diffusion:**



**Consistency approach:**



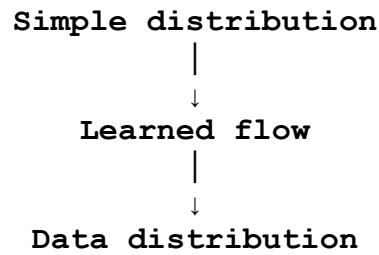
The key objective is therefore **sampling efficiency**, while attempting to preserve the quality associated with iterative diffusion generation.

## 4.6 Flow Matching

Flow matching takes another perspective.

Instead of explicitly learning a discrete sequence of denoising transitions, the objective is to learn a continuous-time vector field that transports samples from a simple source distribution toward the target data distribution.

Conceptually:



The learned vector field defines a trajectory through the probability space.

A conceptual continuous path can be represented as:

$$x_t$$

where  $t \in [0, 1]$ , with a model learning the velocity field that moves the sample along the desired trajectory.

The advantage is that generation can be formulated as solving a continuous transport problem rather than explicitly reproducing a long discrete denoising chain.

## 4.7 Consistency Models vs Flow Matching

These approaches address the same broad engineering problem—**reducing the cost of generative sampling**—but from different perspectives.

| Approach           | Main idea                                    | Primary benefit                      |
|--------------------|----------------------------------------------|--------------------------------------|
| Diffusion          | Iterative denoising                          | Strong generation quality            |
| Consistency Models | Learn consistent mappings along trajectories | Few-step generation                  |
| Flow Matching      | Learn continuous probability transport       | Efficient continuous-time generation |

Therefore, an important research direction is:

quality ↔ sampling steps ↔ latency ↔ compute.

## 4.8 Engineering Interpretation

From an engineering perspective, the comparison should not simply ask:

**Which generative model is best?**

A better question is:

**Which model provides the required generation quality under the available inference latency and compute budget?**

For a production system, relevant evaluation dimensions include:

- generation quality;
- diversity;
- inference latency;
- number of sampling steps;
- GPU utilization;
- memory consumption;
- computational cost;
- robustness.

This is particularly important when deploying generative models under constrained infrastructure.

## 4.9 Q3 Conclusion

**GANs and diffusion models represent two fundamentally different approaches to generative modeling. GANs rely on adversarial optimization between a generator and discriminator and can provide very fast generation after training, but adversarial training may be difficult to stabilize and can suffer from mode collapse. Diffusion models instead learn a reverse denoising process and generally provide a stable framework for high-quality and diverse generation, at the cost of traditionally requiring multiple sampling steps. Consistency models and flow matching address this inference bottleneck from different directions: consistency models target few-step generation through consistent mappings, while flow matching learns a continuous transport field between source and target distributions. The practical objective is therefore to balance generation quality, diversity, latency, sampling cost, and computational resources rather than treating any single generative architecture as universally optimal.**

## 5. Q4 — LoRA/QLoRA, RAG, Long Context, SFT, RLHF, and DPO

### 5.1 Parameter-Efficient Fine-Tuning: LoRA

Full fine-tuning updates all trainable parameters of a pretrained language model. For large models, this can require substantial GPU memory, storage, and computational resources.

**Low-Rank Adaptation (LoRA)** instead freezes the pretrained weights and learns a low-rank update:

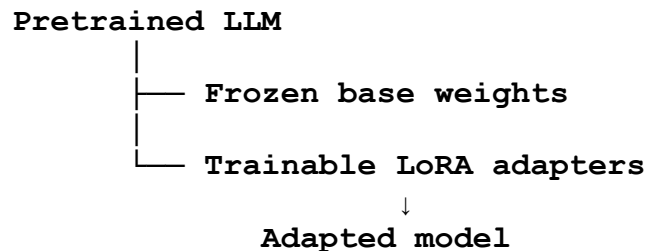
$$W' = W + \Delta W$$

where

$$\Delta W = BA$$

and  $A$  and  $B$  are substantially smaller matrices than the original weight matrix.

Conceptually:

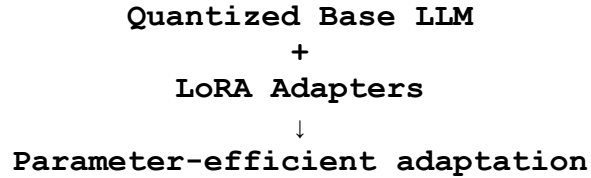


The main advantage is that only a small number of additional parameters need to be trained and stored.

### QLoRA

QLoRA extends this idea by combining LoRA with quantization of the base model. The base model can therefore be represented using lower-precision parameters while LoRA adapters remain trainable.

Conceptually:

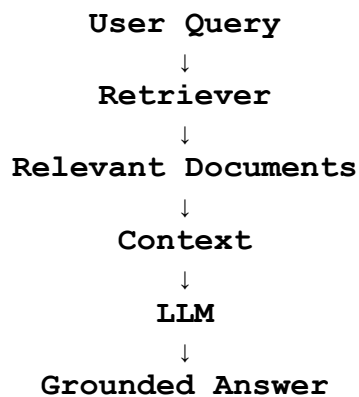


This can reduce memory requirements and make adaptation of larger models more accessible under constrained hardware.

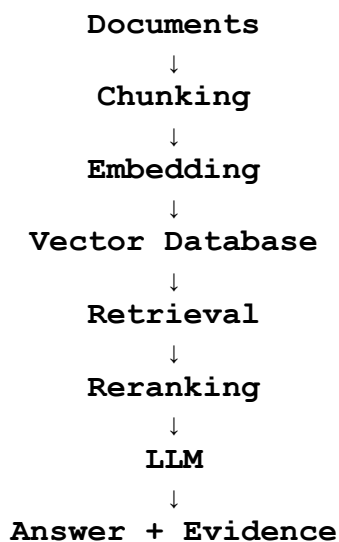
## 5.2 Retrieval-Augmented Generation

Fine-tuning is not the ideal mechanism for storing frequently changing factual knowledge.

**Retrieval-Augmented Generation (RAG)** addresses this by retrieving relevant external knowledge at inference time and providing it to the language model as context.



A typical RAG pipeline is:



RAG therefore changes **where the model obtains knowledge**, rather than changing the model's internal parameters.

## 5.3 RAG vs Fine-Tuning

This distinction is critical.

| Requirement                   | RAG                     | LoRA / Fine-tuning  |
|-------------------------------|-------------------------|---------------------|
| Frequently changing knowledge | <b>Strong fit</b>       | Poor fit            |
| Private documents             | <b>Strong fit</b>       | Possible            |
| Domain terminology            | Possible                | <b>Strong fit</b>   |
| Response style                | Limited                 | <b>Strong fit</b>   |
| New factual knowledge         | <b>Good</b>             | Requires retraining |
| Updating knowledge            | Update retrieval source | Retrain/adapt model |

A useful conceptual distinction is:

**RAG = knowledge adaptation**

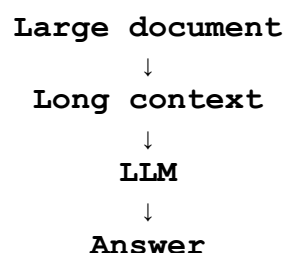
**LoRA = behavior/task adaptation**

These techniques can therefore complement rather than replace one another.

## 5.4 Long Context

Another approach to handling large amounts of information is increasing the model's context window.

A larger context window allows more tokens to be supplied directly to the model:



However, long context does not automatically mean that all information will be equally useful.



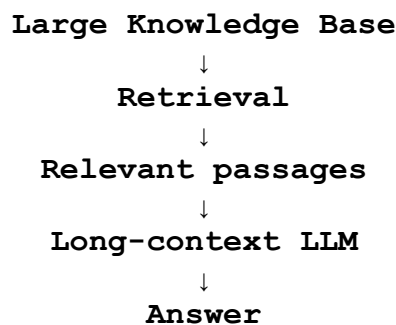
Increasing context can introduce:

- higher memory requirements;
- increased computational cost;
- greater latency;
- irrelevant information;
- attention dilution.

Therefore, simply increasing the context window is not always the optimal solution.

## 5.5 Hybrid Retrieval + Long Context

A practical architecture can combine retrieval with a sufficiently large context window.



The retriever reduces the amount of irrelevant information, while the larger context window allows multiple relevant documents or longer passages to be processed together.

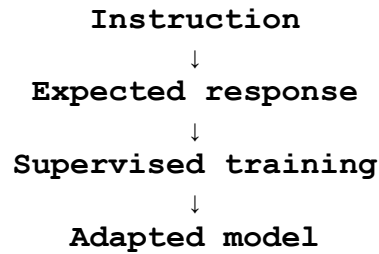
This creates a practical trade-off between:

retrieval quality ↔ context capacity ↔ latency ↔ memory.

## 5.6 Supervised Fine-Tuning

**Supervised Fine-Tuning (SFT)** trains a pretrained model on examples containing desired inputs and outputs.

For example:



The objective is typically to maximize the likelihood of the target response:

$$\mathcal{L}_{SFT} = - \sum_t \log p_{\theta} (y_{t'} | y_{<t} x).$$

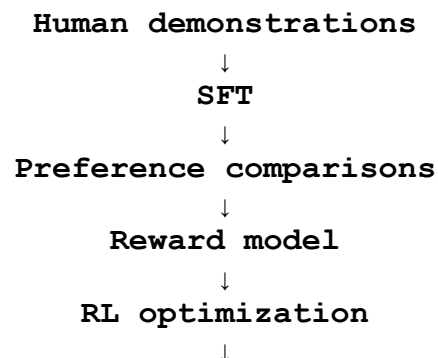
SFT is useful for teaching:

- task-specific behavior;
- response formatting;
- instruction following;
- domain-specific examples.

## 5.7 RLHF

**Reinforcement Learning from Human Feedback (RLHF)** introduces human preference information into the training process.

A simplified pipeline is:



## Aligned model

The key idea is that the model is optimized not only to imitate training examples but also to produce outputs preferred by human evaluators.

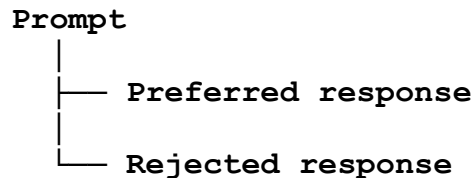
RLHF can be powerful but introduces additional complexity because it requires preference data, reward modeling, and reinforcement-learning optimization.

## 5.8 Direct Preference Optimization

**Direct Preference Optimization (DPO)** provides an alternative approach to preference-based alignment.

Instead of explicitly training a separate reward model and then performing a reinforcement-learning optimization procedure, DPO directly optimizes the language model using preference pairs.

A preference dataset can be represented as:



The model is trained to increase the probability of the preferred response relative to the rejected response while remaining controlled relative to a reference policy.

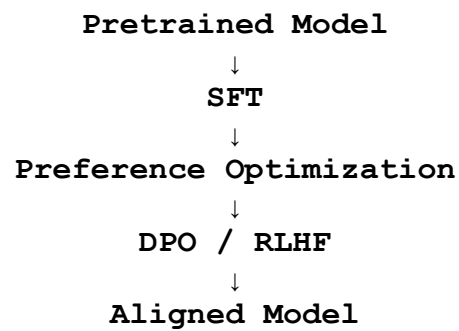
This can make the training pipeline simpler than a conventional RLHF pipeline.

## 5.9 SFT vs RLHF vs DPO

| Method | Main signal                | Main purpose                   | Complexity |
|--------|----------------------------|--------------------------------|------------|
| SFT    | Demonstration data         | Learn desired task behavior    | Lower      |
| RLHF   | Human preferences + reward | Preference alignment           | Higher     |
| DPO    | Preference pairs           | Direct preference optimization | Moderate   |

These approaches are not necessarily mutually exclusive.

A common conceptual progression is:



## 5.10 Engineering Trade-offs

For a production LLM system, the choice among these techniques should depend on the actual problem.

**: If the problem is changing knowledge**

RAG

**If the problem is domain-specific behavior:**

LoRA/QLoRA

**If the problem is instruction following from labeled examples:**

SFT

**If the problem is preference alignment:**

DPO or RLHF

**If the problem is processing very large inputs:**

This leads to an important principle:

**Do not fine-tune a model merely to store information that changes frequently when that information can instead be retrieved at inference time.**

## 5.11 Q4 Conclusion

Modern LLM systems require different adaptation mechanisms for different problems. LoRA and QLoRA provide parameter-efficient methods for adapting pretrained models while reducing the trainable parameter footprint. RAG provides dynamically retrieved knowledge without requiring continual model retraining, making it particularly appropriate for frequently changing information. Long-context processing can accommodate larger inputs but introduces memory, latency, and attention-efficiency trade-offs, making retrieval-based or hybrid approaches attractive in many settings. SFT teaches desired behavior from supervised examples, while RLHF and DPO incorporate preference information for alignment. The engineering decision should therefore be driven by the underlying requirement: RAG for changing knowledge, LoRA/QLoRA for domain or task adaptation, SFT for supervised behavior, and preference optimization when human preferences need to influence model outputs.

## 6. Q5 — AOROA Private LLM Architecture and Evaluation

### 6.1 Problem Context

AOROA Labs operates in the logistics and freight-forwarding domain, where an AI system may need to work with operational and business-related information such as shipment data, freight rates, ERP information, logistics documents, emails, and internal operational records.

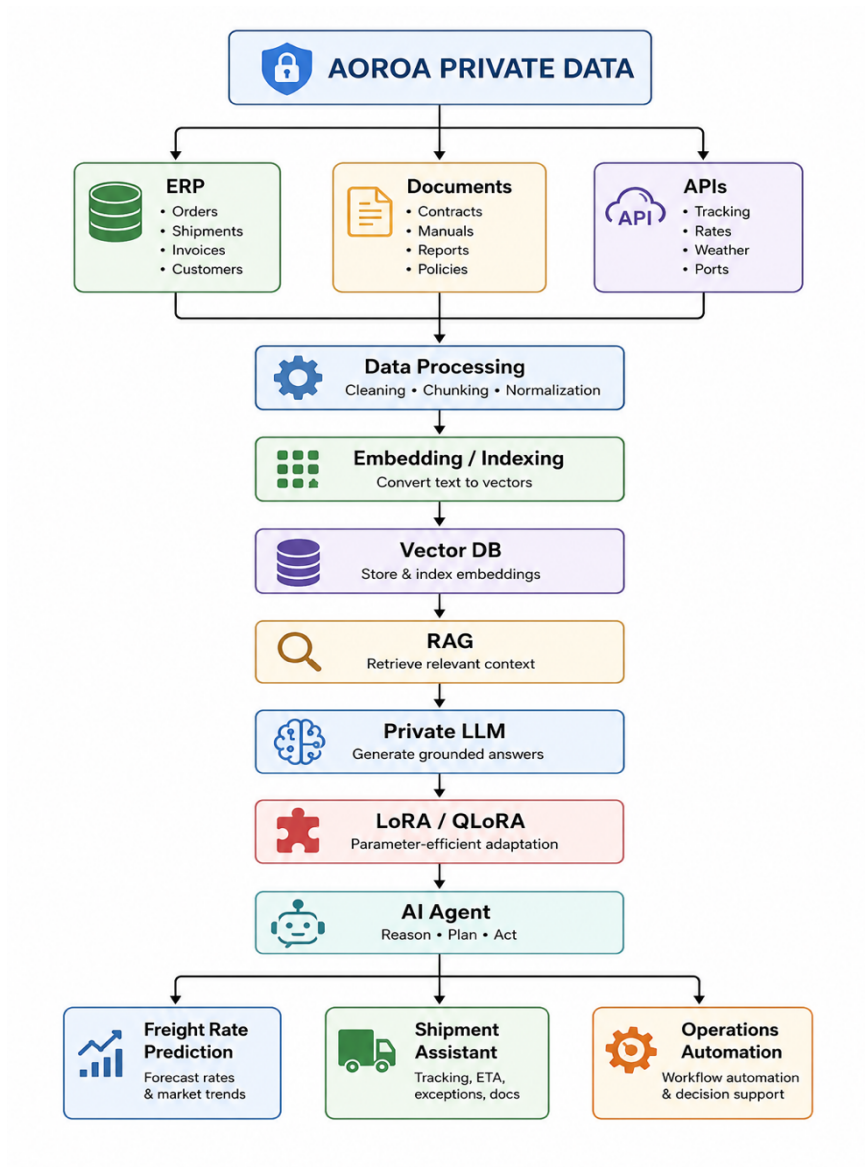
Because such information can be private or commercially sensitive, a suitable architecture should prioritize **data governance, access control, and controlled deployment** rather than sending private company information to external LLM APIs.

The proposed approach is therefore a **privacy-preserving private LLM architecture** in which company data remains within a controlled environment.

**Important:** The architecture described in this section is a **technical proposal for AOROA Labs**, not a claim about AOROA's current production architecture.

## 6.2 Proposed Architecture

The proposed architecture combines a private LLM with retrieval and parameter-efficient adaptation:



The architecture separates two different adaptation problems:

|                            |
|----------------------------|
| RAG = knowledge adaptation |
|----------------------------|

|                                       |
|---------------------------------------|
| LoRA/QLoRA = behavior/task adaptation |
|---------------------------------------|

This distinction is central to the proposed design.

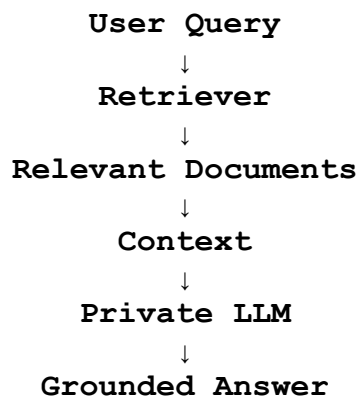
### 6.3 Why RAG?

Logistics information can change frequently. Examples include:

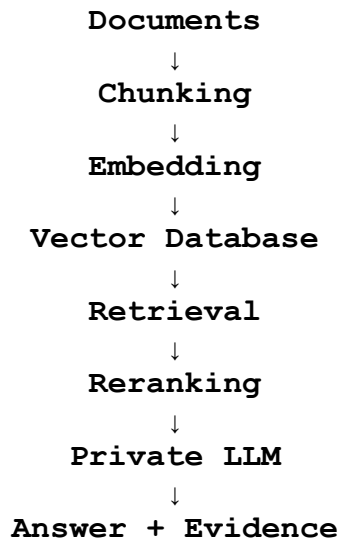
- freight rates;
- shipment status;
- port information;
- customer-specific rules;
- internal SOPs;
- contract information.

Continuously retraining the LLM whenever such information changes would be inefficient.

RAG instead retrieves relevant information at inference time:



A more complete retrieval pipeline is:



Reranking is useful because the highest embedding similarity does not necessarily guarantee the best contextual relevance.

## 6.4 Why LoRA / QLoRA?

If the model needs to learn domain-specific terminology, task formats, or response behavior, parameter-efficient adaptation can be considered.

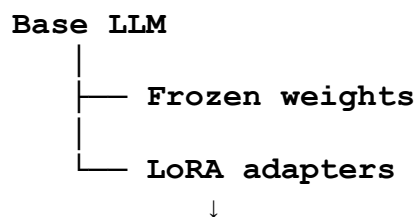
LoRA represents the adapted weight as:

$$W' = W + \Delta W$$

with

$$\Delta W = BA.$$

The pretrained model weights remain frozen while the low-rank matrices are trained.





### Domain-adapted model

QLoRA extends this concept by combining LoRA with quantization of the base model, potentially reducing memory requirements during adaptation.

For a startup environment, this provides a potentially more resource-efficient alternative to full fine-tuning.

## 6.5 RAG + LoRA/QLoRA

RAG and LoRA should not be treated as competing alternatives.

They solve different problems:

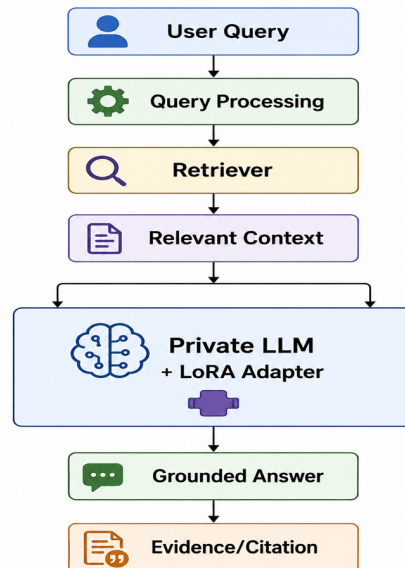
### RAG asks:

Where should the model obtain current domain knowledge?

### LoRA asks:

How should the model behave on domain-specific tasks?

The combined architecture is therefore:



This provides a separation between **knowledge retrieval** and **model adaptation**.

## 6.6 Evaluation Framework

A private LLM should not be evaluated using only a general LLM benchmark.

For AOROA's proposed system, evaluation should cover multiple dimensions:

| Metric                      | What it measures                                      |
|-----------------------------|-------------------------------------------------------|
| Retrieval Recall@K          | Whether relevant evidence is retrieved                |
| Answer Accuracy             | Whether the generated answer is correct               |
| Faithfulness / Groundedness | Whether the answer is supported by retrieved evidence |
| Citation Correctness        | Whether cited evidence actually supports the answer   |
| Hallucination Rate          | Frequency of unsupported claims                       |
| Latency                     | Response time                                         |
| Cost / Resource Usage       | Computational requirements                            |
| Task Completion Rate        | Whether the system completes the intended task        |

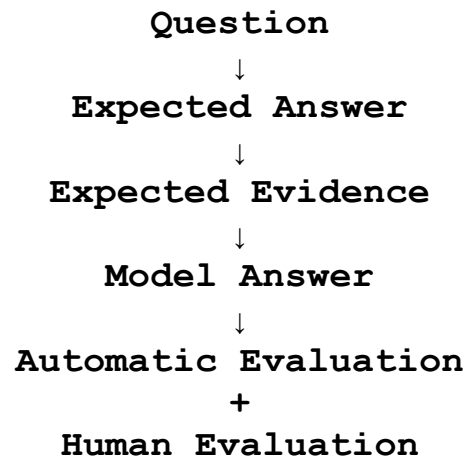
Additional evaluation dimensions can include:

- robustness;
- security/privacy;
- human evaluation.

## 6.7 Controlled Evaluation Dataset

Evaluation should use a controlled internal benchmark rather than exposing private company data to an external LLM API.

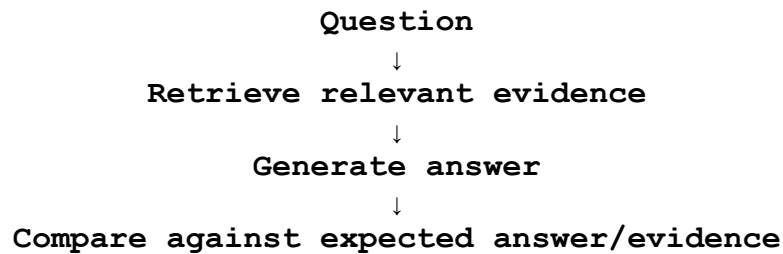
A suitable evaluation dataset can contain:



For example:

**Question:** What is the applicable freight rate for shipment X?

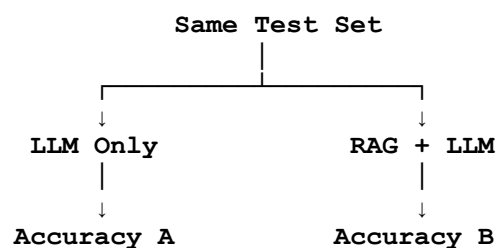
The system should:



This makes the performance of the RAG system measurable.

## 6.8 A/B Evaluation

A controlled A/B experiment can test whether RAG provides measurable benefit.



Hallucination A  
Latency A

Hallucination B  
Latency B

The hypothesis can be expressed as:

$$H_1: Accuracy_{RAG} > Accuracy_{LLM}$$

and:

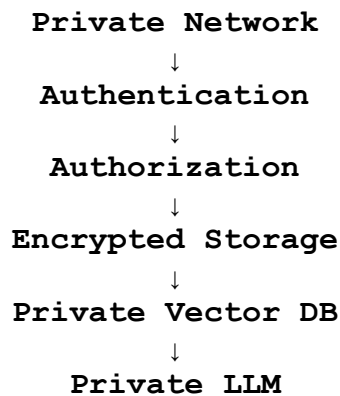
$$H_2: Hallucination_{RAG} < Hallucination_{LLM}.$$

If these differences are observed on the controlled evaluation set, the result provides empirical evidence for the usefulness of RAG.

This is an **experimental hypothesis**, not a causal claim about real-world business outcomes.

## 6.9 Security and Privacy

A private LLM architecture should include appropriate security controls:



Important controls include:

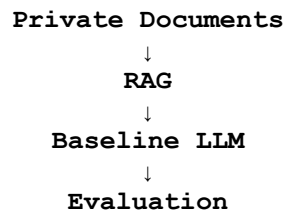
- PII protection;
- access control;
- audit logging;
- encryption;
- data retention policies;
- document-level permissions.

The retrieval layer should also respect document permissions so that users cannot retrieve information they are not authorized to access.

## 6.10 Recommended Development Strategy

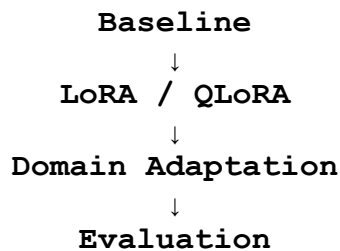
A staged development strategy is preferable to immediately performing extensive fine-tuning.

### Phase 1 — Private RAG Baseline



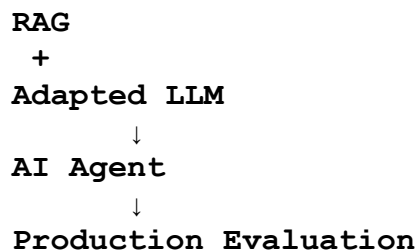
First establish whether retrieval itself improves performance.

### Phase 2 — LoRA / QLoRA



Only introduce parameter-efficient adaptation if the evaluation indicates that domain-specific behavior requires it.

### Phase 3 — Integrated System



This staged approach makes each improvement measurable.

## 6.11 Final Recommendation

For AOROA Labs, I recommend evaluating a:

**Private LLM + RAG + LoRA/QLoRA + domain-specific evaluation**

architecture.

The central design principle is:

**Use RAG when the problem is access to changing knowledge; use LoRA/QLoRA when the problem is domain-specific behavior or task adaptation.**

The proposed system should first establish a private RAG baseline and measure retrieval quality, answer accuracy, groundedness, citation correctness, hallucination, latency, resource usage, and task completion. Parameter-efficient adaptation should then be introduced only where controlled evaluation demonstrates a measurable benefit.

This provides an incremental and evidence-based path from a private knowledge assistant toward a domain-specific logistics AI agent.

## 6.12 Q5 Final Answer

**For AOROA Labs, I would recommend a privacy-preserving private LLM architecture combining Retrieval-Augmented Generation (RAG) with parameter-efficient adaptation such as LoRA/QLoRA. RAG should provide access to frequently changing logistics knowledge from private ERP systems, operational documents, APIs, and internal knowledge bases without requiring continual model retraining. LoRA/QLoRA can be used when domain-specific behavior, terminology, task formatting, or instruction following requires model adaptation. The proposed system should be evaluated using retrieval recall, answer accuracy, groundedness, citation correctness, hallucination rate, latency, computational cost, and task completion. Evaluation should use a controlled internal benchmark containing questions and expected evidence, while private company data should remain within an appropriately controlled environment rather than being sent to external LLM APIs.**

## 7. Overall Technical Conclusion

This report has examined five core areas of modern AI/ML systems that are fundamental to the design and deployment of private, domain-specific language-model solutions for logistics and freight-forwarding applications.

**Q1** established the probabilistic foundation through variational inference, KL divergence, and the evidence lower bound (ELBO). The connection between variational autoencoders and diffusion models demonstrates that diffusion training objectives are not heuristic but rather practical parameterizations of a principled probabilistic framework. This theoretical understanding is essential for developing generative models that can operate effectively within constrained, private environments.

**Q2** analyzed the Transformer architecture with particular attention to inference efficiency. The scaled dot-product attention mechanism, multi-head attention, and KV caching provide the computational backbone for modern LLMs. Grouped-Query Attention (GQA) offers a practical engineering trade-off between attention capacity and memory efficiency — a critical consideration for private deployments where GPU memory, latency, and concurrent user support are constrained. These architectural choices directly affect the feasibility and cost of serving private LLMs in production.

**Q3** compared generative modeling approaches, highlighting the fundamental differences between GANs and diffusion models. While GANs offer fast generation, they suffer from training instability and mode collapse. Diffusion models provide a stable, likelihood-based framework with strong diversity, at the cost of multi-step sampling. Consistency models and flow matching represent promising directions for reducing this inference bottleneck, achieving fewer-step generation while preserving quality. The practical lesson is that the choice of generative architecture must be driven by the specific requirements of the application — quality, diversity, latency, and compute budget — rather than assuming any single approach is universally superior.

**Q4** examined practical methods for adapting and augmenting large language models. The key distinction between RAG (knowledge adaptation) and LoRA/QLoRA (behavior/task adaptation) is central to modern LLM system design. RAG is appropriate for frequently changing information that does not require model retraining, while

LoRA/QLoRA provides parameter-efficient domain adaptation for terminology, task formatting, and response behavior. SFT teaches desired behavior from supervised examples, while RLHF and DPO incorporate human preferences for alignment. The engineering principle is clear: use the right tool for the right problem, and avoid fine-tuning a model merely to store information that changes frequently.

**Q5** synthesized these insights into a concrete architecture for AOROA Labs: a privacy-preserving private LLM combining RAG with LoRA/QLoRA. The proposed system separates knowledge retrieval from model adaptation, allowing each component to be developed, evaluated, and improved independently. The evaluation framework covers retrieval quality, answer accuracy, groundedness, citation correctness, hallucination rate, latency, resource usage, and task completion. Security controls — authentication, authorization, encryption, audit logging, and document-level permissions — ensure that private company data remains within a controlled environment.

**The overarching recommendation** is to develop the system incrementally: start with a private RAG baseline, evaluate its performance thoroughly, and introduce parameter-efficient adaptation (LoRA/QLoRA) only where controlled evaluation demonstrates measurable benefit. This staged, evidence-based approach avoids premature complexity and makes each improvement measurable.

**Final Thought:** The design of private, domain-specific LLM systems is not simply about selecting the largest model or the latest technique. It is about understanding the problem requirements, selecting appropriate architectural components that provide the right quality–cost–latency trade-offs, and evaluating systematically. The principles established in this report — probabilistic foundations, efficient attention architectures, appropriate adaptation mechanisms, and rigorous evaluation — provide a solid foundation for building AI systems that are both capable and trustworthy in logistics and freight-forwarding applications.



## 8. References

### Q1 — Variational Inference, ELBO, and Diffusion Models

1. Kingma, D. P., & Welling, M. (2014). *Auto-Encoding Variational Bayes*. In *Proceedings of the International Conference on Learning Representations (ICLR)*.  
 <https://arxiv.org/abs/1312.6114>
  2. Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). *Variational Inference: A Review for Statisticians*. *Journal of the American Statistical Association*, 112(518), 859–877.  
 <https://arxiv.org/abs/1601.00670>
  3. Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 6840–6851.  
 <https://arxiv.org/abs/2006.11239>
  4. Sohl-Dickstein, J., Weiss, E. A., Maheswaranathan, N., & Ganguli, S. (2015). *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2256–2265.  
 <https://arxiv.org/abs/1503.03585>
- 

### Q2 — Transformer Architecture, Attention, KV Cache, and GQA

5. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008.  
 <https://arxiv.org/abs/1706.03762>
6. Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., & Sanghai, S. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.  
 <https://arxiv.org/abs/2305.13245>
7. Shazeer, N. (2019). *Fast Transformer Decoding: One Write-Head is All You Need*. *arXiv preprint* *arXiv:1911.02150*.  
 <https://arxiv.org/abs/1911.02150>

---

### Q3 — Diffusion Models, GANs, Consistency Models, and Flow Matching

8. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Nets*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 27, 2672–2680.  
<https://arxiv.org/abs/1406.2661>
9. Song, Y., & Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 11895–11907.  
<https://arxiv.org/abs/1907.05600>
10. Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations*. In *Proceedings of the International Conference on Learning Representations (ICLR)*.  
<https://arxiv.org/abs/2011.13456>
11. Song, Y., & Dhariwal, P. (2023). *Improved Techniques for Training Consistency Models*. *arXiv preprint arXiv:2310.02278*.  
<https://arxiv.org/abs/2310.02278>
12. Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow Matching for Generative Modeling*. In *Proceedings of the International Conference on Learning Representations (ICLR)*.  
<https://arxiv.org/abs/2210.02747>

---

### Q4 — LLM Adaptation: LoRA, QLoRA, RAG, SFT, RLHF, DPO

13. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. In *Proceedings of the International Conference on Learning Representations (ICLR)*.  
<https://arxiv.org/abs/2106.09685>
14. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2024). *QLoRA: Efficient Finetuning of Quantized LLMs*. In *Advances in Neural Information Processing Systems (NeurIPS)*.  
<https://arxiv.org/abs/2305.14314>
15. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.  
<https://arxiv.org/abs/2005.11401>
16. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). *Training Language Models to Follow Instructions with Human Feedback*. In *Advances in Neural*

*Information Processing Systems (NeurIPS)*, 35, 27730–27744.  
 <https://arxiv.org/abs/2203.02155>

17. Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. In *Advances in Neural Information Processing Systems (NeurIPS)*.  
 <https://arxiv.org/abs/2305.18290>
- 

## Q5 — Private LLM Architecture and Evaluation

18. Gao, L., & Callan, J. (2022). *A Survey of Retrieval-Augmented Generation*. *arXiv preprint arXiv:2205.11476*.  
 <https://arxiv.org/abs/2205.11476>
19. Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., ... & Liang, P. (2021). *On the Opportunities and Risks of Foundation Models*. *arXiv preprint arXiv:2108.07258*.  
 <https://arxiv.org/abs/2108.07258>
20. Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y., Lundberg, S., Nori, H., Palangi, H., Ribeiro, M. T., & Zhang, Y. (2023). *Sparks of Artificial General Intelligence: Early Experiments with GPT-4*. *arXiv preprint arXiv:2303.12712*.  
 <https://arxiv.org/abs/2303.12712>